Object Oriented Programming

Courtesy:
E Balagurusamy, Yashwant Kanetkar, Programiz,
GeeksforGeeks

Dr. Balu L. Parne

DoCSE, SVNIT Surat.

Managing Console Input / Output Operations

Introduction:

- Every program takes some data as input and generates processed data as output following the familiar **input-process-output** cycle.
- It is important to understand how to provide the input data and present the result in a desired form.
- We have used **cin** and **cout** with the operators **>>** and **<<** for input and output operations. However, we haven't focused on how to control the way the output is printed.
- C++ supports a rich set of I/O functions and operations to print the output.

Introduction:

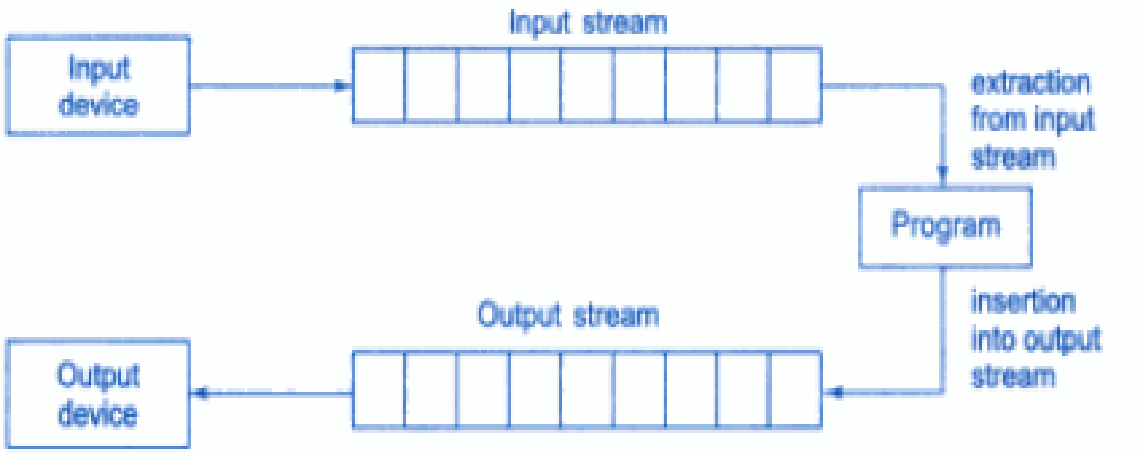
- C++ supports all of C's rich set of I/O functions. We can use them in the C++ programs but we restrained from using them due to two reason:
  - Input / Output methods in C++ supports the concepts of OOP.
  - Input / Output methods in C cannot handle the user-defined data types such as class objects.
- C++ supports a rich set of input/output functions and operations.
- C++ uses the concept of stream and stream classes to implement its I/O operations with the console and disk files.

C++ Streams:

- The I/O system in C++ is designed to work with a wide variety of devices including terminals, disks, and tape drives.
- Although each device is very different, the I/O system supplies an interface to the programmer that is independent of the actual device being accessed. This interface is known as ***stream.***
- A stream is a sequence of bytes.
- The source stream that provides data to the program is called the **input stream.**
- The destination stream that receives output from the program is called the **output stream.**

C++ Streams:

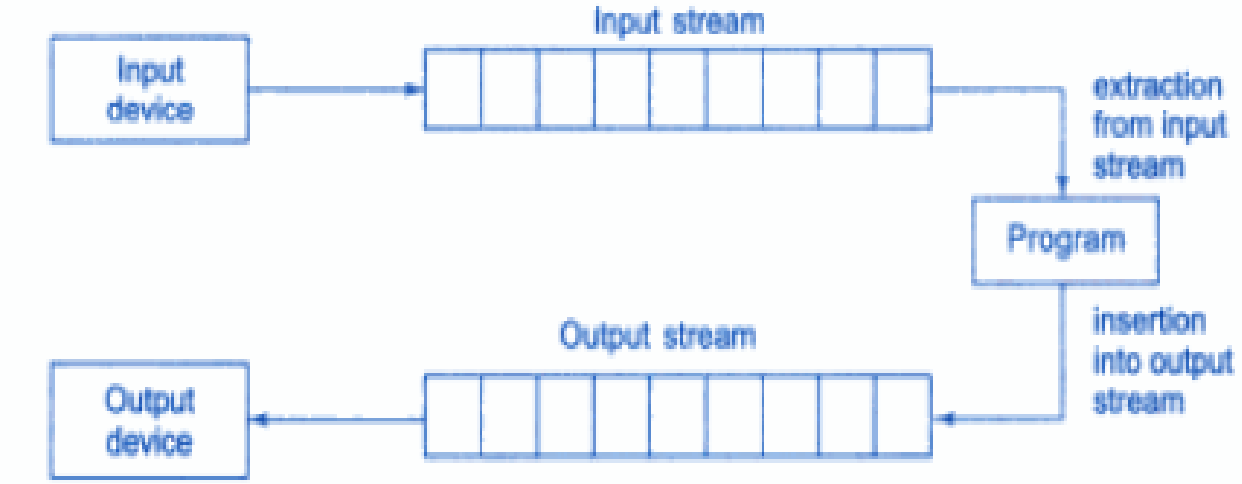
- A stream is a sequence of bytes. It acts either as a source from which the input data can be obtained or as a destination to which the output data can be sent.



- The data in the input stream can come from the keyboard or any other storage device. Similarly, the data in the output stream can go to the screen or any other storage device.

C++ Streams:

- A stream acts as an interface between the program and the input / output device.
- A program extracts the bytes from an input stream and inserts bytes into an output stream.



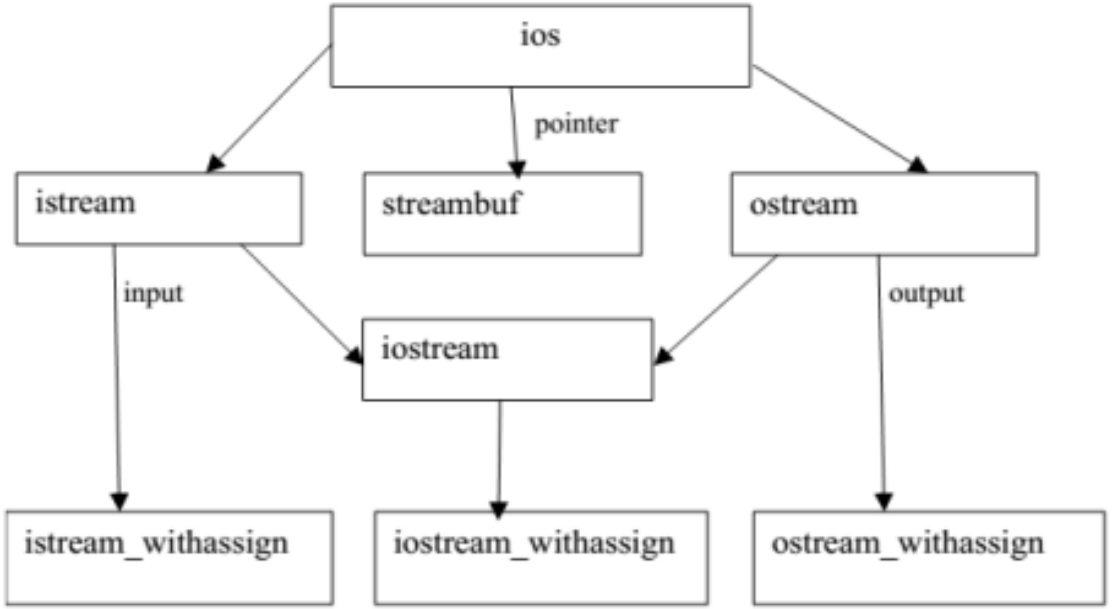
- Keyboard and the screen are default options, we can redirect streams to other devices or files, if required.

C++ Streams:

- C++ contains several pre-defined streams that are automatically opened when a program begins its execution.
- These include **cin** and **cout.**
- **cin** represents the input stream connected to the standard input device (usually the keyboard).
- **cout** represents the output stream connected to the standard output device (usually the screen).
- Keyboard and the screen are default options. **We can redirect streams to other devices or files, if necessary.**

C++ Stream Classes:

- The C++ I/O system contains a hierarchy of classes that are used to define various streams to deal with both the console and disk files. These classes are called **stream classes.**



Stream classes for console I/O operations

- These classes are declared in the header file **iostream.** This file should be included in all the programs that communicate with the console unit.

C++ Stream Classes:

- The **ios** is the **base class** for **istream (input stream)** and **ostream (output stream)** which are in turn **base classes** for **iostream** (input/output stream).
- The class **ios** is declared as the virtual base class so that only one copy of its members are inherited by the **iostream.**
- The class **ios** provides the basic support for formatted and unformatted I/O operations.
- The class **istream** provides the facilities formatted and unformatted input while the class **ostream** (through inheritance) provides the facilities for formatted output.
- The class **iostream** provides the facilities for handling both input and output streams.

C++ Stream Classes:

ios (General input/output stream class)	• Contains basic facilities that are used by all other input and output classes • Also contains a pointer to a buffer object (**streambuf** object) • Declares Constants and functions that are necessary for handling formatted input and output operations
istream (Input stream)	• It inherits the properties of **ios** • It declares input functions such as get(), getline(), and read() • It contains overload extraction operator >>

C++ Stream Classes:

ostream (Output Stream)	• It inherits the properties of **ios**. • It declares input functions such as **put()** and **write()**. • It contains an overload extraction operator **<<**.
iostream (Input/output stream)	• Inherits the properties of **ios stream** and **ostream** through multiple inheritances and thus contains all the input and output functions.
Streambuf	• It provides an interface to physical devices through buffers. • It acts as a base for filebuf class used ios file.

Unformatted I/O Operations:

- **cin** is an object of type **istream** and **cout** an object of type **ostream.**
- These two objects have been declared in the **namespace**-std.
- Using objects **cin** and **cout** for the input and the output of data of various types is possible because of overloading of operator >> and << to recognize all the basic C++ types.
- The **>>** operator is overloaded in the **istream** class and **<<** is overloaded in the **ostream** class.
- The general format for reading data from the keyboard:

cin >> var1 >> var2 >> …. >> var_n;

Unformatted I/O Operations:

- The general format for reading data from the keyboard:

cin >> var1 >> var2 >> …. >> var_n;

- Here, **var1, var2, ……, var_n** are the variable names that are declared already.
- The input data must be separated by white space characters and the data type of user input must be similar to the data types of the variables which are declared in the program.
- The **operator >>** reads the data character by character and assigns it to the indicated location.
- **Reading of variables terminates when white space occurs or character type occurs that does not match the destination type.**

Unformatted I/O Operations:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int data;
    char val;
    // Input the data
    cin >> data;
    cin >> val;
    // Print the data
    cout << data << "    " << val;
    return 0;
}
```

istream class functions:

- This class is responsible for handling input stream.
- The istream class provides a range of functions for handling characters, strings, and objects, including **get, getline, read, ignore, putback, and cin.**
- These functions enable developers to manipulate input in various ways.
- For example, the **get** function allows developers **to read a single character** from the input stream, while the **getline** function **reads an entire line of text and stores it in a string object.** The **read** function is used to **read a specific number of characters** from the input stream and store them in a buffer. Overall, the **istream class** is an essential component of input stream handling in C++.

put() and get() functions:

- The class **istream** and **ostream** have predefined functions **get()** and **put()**, **to handle single character input and output operations.**
- The function get() can be used in two ways, such as **get(char*)** and **get(void)** to fetch characters including blank spaces, newline characters, and tab.
- The function **get(char*)** assigns the value to a variable and **get(void)** to return the value of the character.

put() and get() functions:

```
char data;

// get() return the character value and assign to data variable
data = cin.get();

// Display the value stored in data variable
cout.put(data);
```

```
char c;

// directly assign value to c
cin.get(c);

// Display the value stored in c variable
cout.put()
```

put() and get() functions:

```
#include <iostream>
using namespace std;
int main()
{
    char a;
    cout<<"Enter the characters" << endl;
    cin.get(a);
    cout << a;
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main()
{
char u;
cout<<"Enter the characters " << endl;
cin.get(u);
cout.put(u);
cout<<endl;
cout.put(98);
return 0;
}
```

ostream class functions:

- The **ostream** class contains the declarations of flags, member functions and overloaded operator functions of operations related to output.
- The **ostream** class is responsible for working with the output stream and provides all the necessary functions for managing chars, strings, and objects such as **put, write, cout** etc.
- Apart from the **overloaded insertion operator (<<)**, member functions of this class include eof, clear, fill, etc.

Character I/O with get() and put():

```
#include <iostream>
using namespace std;
// Driver Code
int main()
{
    char data;
    int count = 0;
     cout << "Enter Data: ";
     // Get the data
    cin.get(data);
    while (data != '\n')
    {
        // Print the data
        cout.put(data);
        count++;
         // Get the data again
        cin.get(data);
    }
    cout<<"\n Number of Characters = " << count << endl;
    return 0;
}
```

When we type a line of input, the text is sent to the program as soon as we press the RETURN key. The program then reads one character at a time using the statement **cin.get(data);** and display it using the statement **cout.put(data);**. The process is terminated when the newline character is encountered.

getline() and write() Functions:

- In C++, the function **getline() and write()** provide a more efficient way to handle line-oriented inputs and outputs.
- **getline()** function reads the complete line of text that ends with the new line character.
- The **getline()** function reads a whole line of text that ends with a newline character (transmitted by the Return Key).
- This function can be invoked using the **cin** object.

```
cin.getline(variable_to_store_line, size);
```

getline() and write() Functions:

```
cin.getline(variable_to_store_line, size);
```

- This function call invokes the function getline() which reads character input into the variable line.
- **The reading is terminated as soon as either the newline character '\n' is encountered or size-1 characters are read (whichever occurs first).**
- The newline character is read but not saved. Instead, it is replaced by the null character.

getline() and write() Functions:

- The **write()** function displays the entire line in one go and its syntax is similar to the **getline()** function only that here **cout object** is used to invoke it.

```
cout.write(variable_to_store_line, size);
```

- The key point to remember is that the write() function does not stop displaying the string automatically when a NULL character occurs.
- If the size is greater than the length of the line then, the write() function displays beyond the bound of the line.

getline() and write() Functions:

```
#include<iostream>
using namespace std;
int main()
{
    int size =20;
    char city[20];
    cout<<"Enter city name : \n " <<endl;
    cin>>city;
    cout<<"City name is : " << city << endl;

    cout<<"Enter city name again: \n " << endl;
    cin.getline(city, size);
    cout<<"City name now is : " << city << endl;

    cout<<"Enter another city name : \n " << endl;
    cin.getline(city, size);
    cout<<"New city name is : " << city << endl;

    return 0;
}
```

Formatted Console I/O Operations:

- C++ supports a number of features that could be used for formatting the output. These features include:
  - **ios** class functions and flags
  - Manipulators
  - User-defined output functions
- The **ios** class contains a large number of member functions that would help us to format the output in a number of ways.
- Manipulators are special functions that can be included in the I/O statements to alter the format parameters of a stream.

Formatted Console I/O Operations:

Function	Task
width()	To specify the required field size for displaying an output value.
precision()	To specify the number of digits to be displayed after the decimal point of a float value.
fill()	To specify a character that is used to fill the unused portion of a field.
setf()	To specify format flags that can control the form of the output display (Such as left justification and right justification).
unsetf()	It is used to clear the flags specified.

Formatted Console I/O Operations:

- Manipulators are special functions that can be included in the I/O statements to alter the format parameters of a stream.

Manipulators	Task
setw()	It is used to specify the required field size for displaying an output value.
setprecision()	It is used to specify the number of digits to be displayed after the decimal point of a float value.
setfill()	It is used to specify a character that is used to fill the unused portion of a field.
setiosflags()	It is used to specify format flags that can control the form of the output display.
resetiosflags()	It is used to clear the flags specified.

Using the width() formatted output function:

- Using this function, we can specify the width of a value to be displayed in the output at the console.

```
cout.width(w);
```

- Where, w is the field width (number of columns). The output will be printed in a field of w characters wide at the right end of the field.

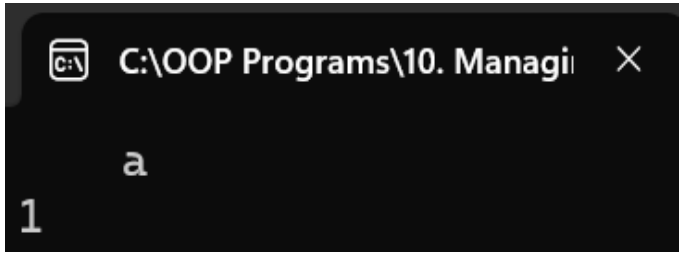
```
cout.width(5);
cout << 456 << 12 << "\n";
```

- The width function can specify the field width for only one item(the item that follows immediately). After printing one item (as per the specifications) it will revert back to the default.
- **The field width should be specified for each item separately.**

```
cout.width(5);
cout << 543;
cout.width(5);
cout << 12 << "\n";
```

Using the width() formatted output function:

```
#include<iostream>
using namespace std;
int main()
{
    char ch = 'a';
    cout.width(5);
    cout<<ch <<"\n";
    int i = 1;
    cout<<i;
}
```



C++ never truncates the values and therefore, if the specified field width is smaller than the size of the value to be printed, C++ expands the field to fit the value.

Setting Precision: precision()

- By default, the floating numbers are printed with six digits after the decimal point.
- We can specify the number of digits to be displayed after the decimal point while printing the floating point numbers.
- This can be done by precision member function.

```
cout.precision(d)
```

- Where d is the number of digits to the right of the decimal point.

Setting Precision: precision()

```
cout.precision(3);
cout<<sqrt(2) <<"\n"; //1.141 (truncated)
cout<<3.14159 <<"\n"; //3.142 (rounded to the nearest cent)
cout<<2.50032 <<"\n"; //2.5 (no trailing zeros)
```

- Unlike the function width(), precision() retains the setting in effect until it is reset.
- We can also combine the field specification with the precision setting.

```
cout.precision(2);
cout.width(5);
cout << 1.2345
```

Setting Precision: precision()

- Before calling the precision() function, we should call the setf() function to set the flag to display a fixed number of digits after the decimal in a floating-point number by using the flag values, **ios::fixed and ios::floatfield**, otherwise, **directly calling the precision() function would only fix the total number of digits to display in the output.**

```
#include<iostream>
using namespace std;
int main()
{
    float d = 12.3458998f;
    cout<< "The floating value in variable d : " << d <<"\n";
    cout.setf(ios::fixed, ios::floatfield);
    cout.precision(3);
    cout<< "The floating value in variable d, after calling the precision() function : " << d;
}
```

Filling and Padding: fill()

- We have been printing the values using much larger field widths than required by the values.
- The unused positions of the field are filled with white spaces, by default.
- We can use the **fill()** function to fill the unused positions by any desired character.

```
cout.fill(ch);
```

```
cout.fill('*');
cout.width(10);
cout << 5250 << "\n";
```

- Where, ***ch*** represents the character which is used for filling the unused positions. Like, **precision(), fill()** stays in effect till we change it.

Formatting Flags, Bit-fields and setf():

- When the function width() is used, the value is printed right-justified in the field width created. But usual practice is to print the text left-justified.
- *How do we get a value printed left-justified?*
- The **setf()** (stands for set flags), a member function of the **ios** class, can provide answers to these and many other formatting questions.

```
cout.setf(arg1, arg2);
```

- The ***arg1*** is one of the formatting flags defined in the class **ios**. The formatting flag specifies the format action required for the output.
- Another **ios** constant, ***arg2,*** known as bit field specifies the group to which the formatting flag belongs.

Formatting Flags, Bit-fields and setf():

Format	Flag	Bit Field
Left justification	ios::left	ios::adjustfield
Right justification	ios::right	ios::adjustfield
Padding after sign and base	ios::internal	ios::adjustfield
Scientific notation	ios::scientific	ios::floatfield
Fixed point notation	ios::fixed	ios::floatfield
Decimal base	ios::dec	ios::basefield
Octal base	ios::oct	ios::basefield
Hexadecimal base	ios::hex	ios::basefield

Displaying Trailing Zeros and Plus Sign:

- If we print the numbers 10.75, 25.00 and 15.50 using a field width of, say, eight positions, with two digits precision.
- Note that the trailing zeros in the second and third items have been truncated.
- In certain situations, such as a list of prices of items or the salary statement of employees, require trailing zeros to be shown. The above values will look better if they are printed as 10.75, 25.00, 15.50.
- The **setf()** can be used with the flag **ios::showpoint** as a single argument to achieve this form of output.

Displaying Trailing Zeros and Plus Sign:

- The **setf()** can be used with the flag **ios::showpoint** as a single argument to achieve this form of output.

```
cout.setf(ios::showpoint); //display trailing zeros
```

- Would cause cout to display trailing zeros and trailing decimal point.
- Similarly, a plus sign can be printed before a positive number using the following statement:

```
cout.setf(ios::showpos); //display + sign
```

Displaying Trailing Zeros and Plus Sign:

```
cout.setf(ios::showpoint);
cout.setf(ios::showpos);
cout.precision(3);
cout.setf(ios::fixed, ios::floatfield);
cout.setf(ios::internal, ios::adjustfield);
cout.width(10);
cout<<275.5 <<"\n";
```

- The flags such as **showpoint** and **showpos** do not have any bit fields and therefore are used as single arguments in **setf().**
- The flags set by **setf()** remain effective until they are reset or unset.
- A format flag can be reset any number of times in a program.
- We can apply more than one format controls jointly on an output value.
- The setf() sets the specified flags and leaves others unchanged.

Managing Output With Manipulators:

- The header file ***iomanip*** provides a set of functions called *manipulators* which can be used to manipulate the output formats.
- They provide same features as that of the **ios** member functions and flags.
- Some manipulators are more convenient to use than their counterparts in the class **ios.**
- For example, two or more manipulators can be used as a chain in one statement as shown below:

```
cout<<manip1 <<manip2 << manip3 << item;
cout<<manip1 <<item1 << manip2 << item2;
```

Formatted Console I/O Operations:

- Manipulators are special functions that can be included in the I/O statements to alter the format parameters of a stream.

Manipulators	Task
setw()	It is used to specify the required field size for displaying an output value.
setprecision()	It is used to specify the number of digits to be displayed after the decimal point of a float value.
setfill()	It is used to specify a character that is used to fill the unused portion of a field.
setiosflags()	It is used to specify format flags that can control the form of the output display.
resetiosflags()	It is used to clear the flags specified.

Managing Output With Manipulators:

Manipulator	Function
setw(int n)	To set the field width to *n*
setbase	To set the base of the number system
setprecision(int p)	The precision is fixed to *p*
setfill(char f)	To set the character to be filled
setiosflags(long l)	Format flag is set to *l*
resetiosflags(long l)	Removes the flags indicated by *l*
endl	Gives a new line
skipws	Omits white space in input
noskipws	Does not omit white space in the input
ends	Adds null character to close an output string
flush	Flushes the buffer stream
lock	Locks the file associated with the file handle
ws	Omits the leading white spaces present before the first field
hex, oct, dec	Displays the number in hexadecimal or octal or in decimal format

Managing Output With Manipulators:

- Some examples of manipulators are given below:

```
cout << setw(10) <<12345;
//prints the value 12345 right-justified in a field width of 10 characters
cout<<setw(10) << setiosflags(ios::left) << 12345;
//prints the output left justified
```

- One statement can be used to format output for two or more values.

```
cout<<setw(5) <<setprecision(2) << 1.2345
    <<setw(10)<<setprecision(4) <<sqrt(2)
    <<setw(15)<<setiosflags(ios::scientific)<<sqrt(3);
cout<<endl;
```

- Will print all the three values in one line with the field sizes of 5, 10, and 15 respectively. Each output is controlled by different sets of format specifiers.

C++ Output Formatting Manipulators

Use manipulators to control field width, alignment, fill characters, precision, and number bases.

setw(n)

```
cout << setw(10) << 25;
```

Sets the minimum width of the next output field.

setprecision(p)

```
cout << setprecision(3)
<< x;
```

Controls floating-point precision. With fixed, p means digits after the decimal point.

setfill(ch)

```
cout << setfill('*') <<
setw(8) << 25;
```

Sets the character used to fill unused field positions.

fixed

```
cout << fixed << x;
```

Displays floating-point values in fixed-point notation.

left / right

```
cout << right << setw(8)
<< 25;
```

Aligns output to the left or right within the field.

setbase(base)

```
cout << setbase(16) <<
255;
```

Sets the integer base: 8 (octal), 10 (decimal), or 16 (hexadecimal).

Header: #include <iomanip> | Most formatting settings remain active until changed; setw() usually affects only the next field.

C++ Stream and Formatting-Control Manipulators

These manipulators control formatting flags, whitespace handling, line breaks, and output buffering.

setiosflags(flags)

```
cout << setiosflags(ios::showpos) << 25;
```

Enables selected formatting flags; showpos displays + for positive values.

resetiosflags(flags)

```
cout << resetiosflags(ios::showpos) ;
```

Clears the specified formatting flags.

endl

```
cout << "Hello" << endl;
```

Inserts a newline and flushes the output stream.

ws

```
cin >> ws;
```

Consumes leading whitespace characters from the input stream.

ends

```
cout << ends;
```

Inserts a null character ('\\0') into the output stream.

flush

```
cout << flush;
```

Forces buffered output to be sent to the output destination.

Header: #include <iostream> and #include <iomanip> as required. Note: skipws is the default for formatted input; noskipws disables it.

User Defined Manipulators:

- We can design our own manipulators for certain special purposes. The general form for creating a manipulator without any argument is:

```
ostream &manipulators (ostream &output)
{
    .....
    .....
    .....//code
    return output;
}
```

```
ostream & unit (ostream & output)
{
    output <<"inches";
    return output;
}

cout<<36 <<unit;
```

User Defined Manipulators:

- We can also create manipulators that could represent a sequence of operations.

```
ostream & show (ostream & output)
{
    output.setf(ios::showpoint);
    output.setf(ios::showpos);
    output<<setw(10);
    return output;
}
```

- This function defines a manipulator called **show** that turns on the flags **showpoint** and **showpos** declared in the class **ios** and sets the field width to 10.

Thank You.